

Agentic Paper trading

Last amended: 25th Sep, 2026

My folder: D:\OneDrive\Documents\crewai>manuals

GitHub: [Link is here](#)

Some paper trading platforms:

TradeZero:

TradeZero's paper trading platform gives you access to real-time NASDAQ Basic market data, advanced trading tools, and full platform functionality for one month. Use this period to test your strategies, explore key features, and get familiar with the platform before moving to a live account.

Alpaca:

Refer here for details on paper trading. Trade only as per US time.

✅ No, Alpaca does not have a fixed time limit or expiration date on its free paper trading usage.

💡 Paper Trading Details

- **Duration:** Free and unlimited with no expiration time.
- **Resetting:** Reset your simulated balance whenever you want. 📌 Alpaca
- **Environment:** Real-time or 24/5 market simulation. 📌 Alpaca +1

⚠️ Rate and Data Limits

- **API Calls:** Limited to 200 requests per minute on the free Basic plan. 📌 Alpaca +1
- **Market Data:** Free tier includes basic SIP or IEX data feeds, with real-time upgrades optional. 📌 Alpaca Community ... +1

Paper vs Live

Feature	Paper	Live
Eligibility	✓	✓
API Access	✓	✓
Free IEX Real Time Data	✓	✓
MFA	✓	✓
Margin Trading	✓	✓
Short Selling	✓	✓
Premarket/After Hours Trading	✓	✓
Borrow Fees	➡ (Coming Soon!)	✓

Paper trading in Indian Stocks:

To deploy **AI agents** for paper trading in the Indian stock market (NSE/BSE), you need platforms that provide robust **REST APIs** or **WebSockets** for execution, or platforms with built-in **sandbox/virtual environments**.

Because Indian regulators require stringent compliance for live algorithmic trading, using these APIs to connect your AI models (like Python scripts utilizing LLMs, LangChain, or custom machine learning agents) to paper trading environments is the safest way to test.

1. Pure Paper Trading Simulator APIs (Developer-First)

These platforms are dedicated simulators that offer APIs specifically designed to replicate the Indian stock market without needing a registered broker account.

- **MegaBull:** MegaBull provides a completely **free paper trading API** for Indian markets. It features standard REST endpoints that allow your AI agents to programmatically place virtual orders, manage open positions, track holdings, and pull real-time market watch data. It is ideal for developers because it provides an OpenAPI spec to test code workflows directly.
- **AlgoTest:** Primarily known as a rule-based backtesting platform, **AlgoTest** includes a comprehensive **Forward Test (Paper Trading)** engine. You can feed signals generated by your AI agents into the platform via webhooks or Python interfaces to simulate complex execution (including multi-leg option strategies, bracket orders, and stop-losses) against real-time market data.

2. Mainstream Indian Brokers with API Sandboxes / Virtual Modes

If your long-term goal is to transition your AI agents from paper trading to live trading, using an actual broker's developer sandbox is highly recommended. The API syntax remains identical when you switch from virtual to live capital.

3. Middleware & AI-Native Trading Bridges

If you do not want to code the entire raw API architecture yourself, you can use middleware that acts as a bridge between your AI code and Indian brokers.

- **OpenAlgo:** A free, **open-source, self-hosted algorithmic trading platform** built on Python. You can run OpenAlgo locally, write your AI agents in Python, and use OpenAlgo's unified API layer to route paper trades or live trades across 35+ Indian brokers seamlessly.
- **Tradetron:** A no-code/low-code engine where you can deploy strategies in a **virtual paper execution mode**. You can hook your AI agent's logic up to Tradetron via webhooks so that when your AI decides to buy or sell, Tradetron handles the virtual market execution on Indian exchanges.
- **Aalsi Trader:** An emerging **AI-powered algo trading platform** specifically for India. It comes with built-in AI agents that automate stock and F&O trading using ₹10 lakh of virtual capital before you link it to a live broker account.

How to set up your AI Agent workflow:

1. **Data & Logic:** Build your AI agent in Python (using libraries like crewai, langchain, or autogen paired with an LLM like Claude or GPT-4 to analyze market sentiment or data arrays). [1]
2. **Execution Gateway:** Generate an API key from **MegaBull** (for pure paper) or a broker like **Upstox/Angel One** (using their sandbox environment).
3. **Order Placement:** Have your AI agent format decisions into standard JSON payloads and send POST requests to the API endpoints to execute trades in the virtual environment.

Top Python Libraries for Quant Modelling

Depending on whether you want a traditional mathematical approach or an advanced machine learning framework, you should consider these specialized libraries:

1. **vectorbt** (Best for Speed & Optimization)

- **How it works:** Operates entirely on **NumPy** and **Numba** to treat trading data as multi-dimensional matrices.
- **Decision generation:** It can evaluate millions of parameter combinations in seconds to find optimal buy/no-buy triggers.
- **Best for:** Rapid backtesting of mathematical logic and high-frequency matrix operations.

2. **FinRL** (Best for AI & Deep Learning)

- **How it works:** A Reinforcement Learning (RL) framework developed specifically for automated financial trading.
- **Decision generation:** Trains AI "agents" on your historical OHLCV data to dynamically learn when to buy, hold, or sell based on reward functions.
- **Best for:** Cutting-edge quantitative models incorporating deep learning.

3. backtrader (Best for Event-Driven Strategy Rules)

- **How it works:** A mature, object-oriented, event-driven framework where you write explicit rules (e.g., if Condition A and B are met, trigger a buy order).
- **Decision generation:** Iterates through time step-by-step to simulate real-time market execution and track portfolio metrics.
- **Best for:** Coding clear, systematic logic combining structural indicators with custom risk controls.

Minimal Workflow Example (using vectorbt)

Here is a conceptual look at how simple it is to generate automated buy/no-buy decisions using vectorbt:

```
import vectorbt as vbt
import pandas as pd

# 1. Load your historical OHLCV data into a Pandas DataFrame
data = pd.read_csv("historical_ohlc.csv", parse_dates=True,
index_col="Date")

# 2. Define quantitative logic (e.g., Fast moving average crossing a Slow
moving average)
fast_ma = vbt.MA.run(data['Close'], window=10)
slow_ma = vbt.MA.run(data['Close'], window=50)

# 3. Generate entry (Buy) and exit (No Buy/Sell) triggers
entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

# 4. Run the quantitative simulation
portfolio = vbt.Portfolio.from_signals(data['Close'], entries, exits,
fees=0.001)
print(portfolio.stats())
```

Crucial Quant Considerations

- **Garbage In, Garbage Out:** Quant models are highly sensitive to data quality. Ensure your OHLCV data handles corporate actions (like stock splits and dividends) correctly.
- **Overfitting Risk:** If you tweak your parameters too much to perfectly fit historical data, the model will likely fail in live markets. Always separate your data into **In-Sample** (training) and **Out-of-Sample** (testing) periods.

Python libraries for financial indicators

TA-Lib is the top industry-standard Python library for computing technical and financial market indicators. Here is a list of top python libraries for calculating financial indicators:

TA-Lib

- Fast C-based core
- 150+ indicators (RSI, MACD, Bollinger Bands)
- Best for high-performance trading systems

Pandas-TA

- Built natively on Pandas
- 130+ indicators
- Best for readable data pipelines

QuantStats

- Computes risk/return metrics
- Generates portfolio reports
- Best for performance analytics

+++++